Engineering Report

Laravel API Telemetry, Metrics, and Controlled Anomaly Monitoring

This report documents the engineering design behind the Laravel telemetry pipeline, Prometheus metrics exposure, Grafana observability dashboard, and the controlled anomaly experiment used to generate incident-like data. The goal was to create signals that are realistic enough for ML-driven anomaly detection and practical enough for operational triage.

1. System Overview

The API was instrumented so every request emits both structured logs and Prometheus metrics. The endpoints intentionally mix healthy, slow, invalid, and failing behaviors. Normal responses come from /api/normal, deterministic and hard-slow latency comes from /api/slow, system failures come from /api/error, database behavior comes from /api/db, and validation failures come from /api/validate. This creates a telemetry stream that contains both transport-level failures and silent degradations.

The key incident-oriented behavior is the hard-slow mode. Requests to /api/slow?hard=1 still return HTTP 200, but the middleware and centralized classifier mark them as TIMEOUT_ERROR when latency crosses 4000 ms. That makes the dataset useful for detecting brownouts rather than only obvious crashes.

2. Log Schema Design

Each log record follows one stable JSON schema so the dataset can be exported directly to logs.json without downstream reshaping. Fields were chosen to support correlation, aggregation, and later feature engineering:

timestamp and request_id provide traceability across logs and metrics. method, path, query, route_name, and status_code identify the request. latency_ms and error_category are the primary anomaly features. severity helps operational filtering. client_ip and user_agent support clustering and traffic analysis. payload_size_bytes and response_size_bytes help identify size-driven regressions. build_version and host support deployment and node-level correlation. exception_class and message preserve fault context without depending on raw stack traces.

The schema is intentionally flat and stable. Missing values remain present as null instead of disappearing, which prevents schema drift in analytics and machine-learning pipelines.

3. Metrics Engineering

Metrics follow RED conventions: http_requests_total, http_errors_total, and http_request_duration_seconds as a histogram. Labels are bounded to method, normalized path, status, and error_category. High-cardinality values such as request_id, raw query strings, payloads, or client identifiers are intentionally excluded so Prometheus remains usable and Grafana queries stay fast.

Histogram buckets were tuned to the actual endpoint behavior instead of copied from defaults: 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10, +Inf. These boundaries separate fast healthy traffic, moderate latency, the standard 5-second slow path, and the 5 to 7 second hard-slow anomaly path. That makes P50, P95, and P99 meaningful in Grafana.

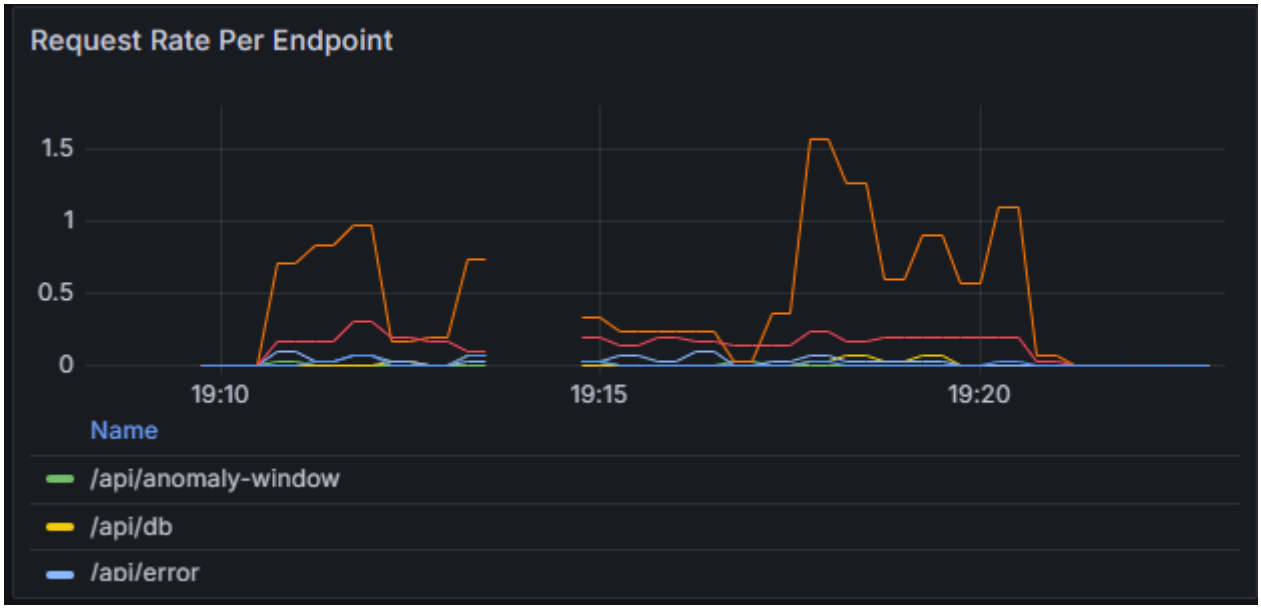
4. Controlled Anomaly Design

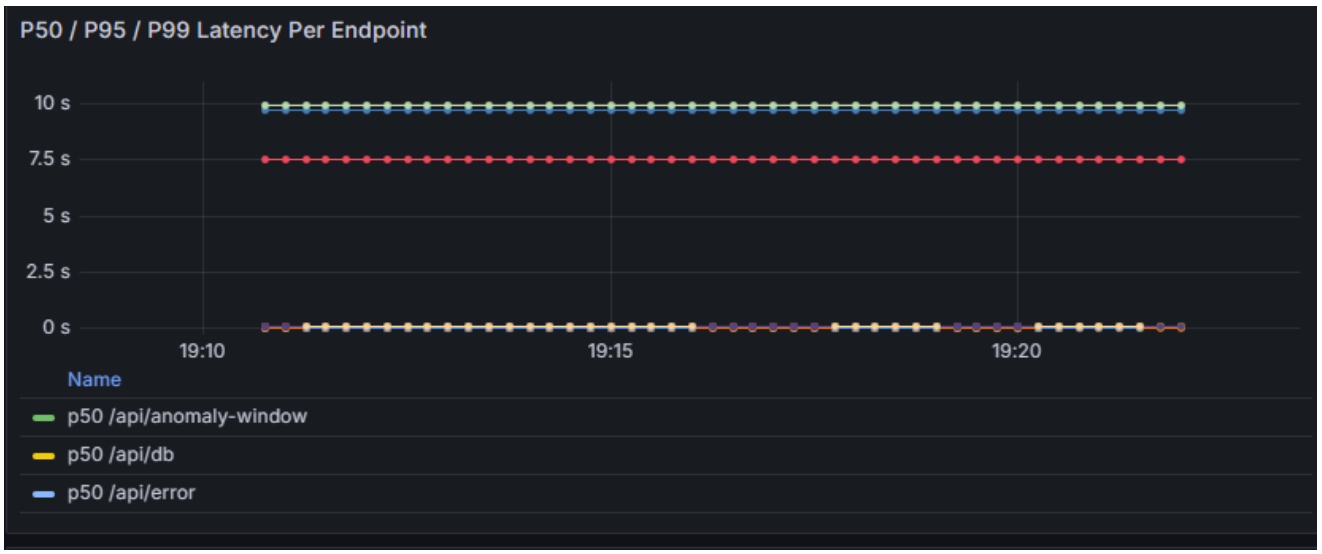
The Python traffic generator produces a mixed workload and then injects a clearly bounded anomaly window. Ground truth is exported to ground_truth.json with anomaly_start_iso, anomaly_end_iso, anomaly_type, and expected_behavior. The selected anomaly type is a latency spike: during the anomaly window, hard-slow requests occupy a much larger share of traffic. This is visible in percentile latency, timeout-category errors, and the anomaly marker metric.

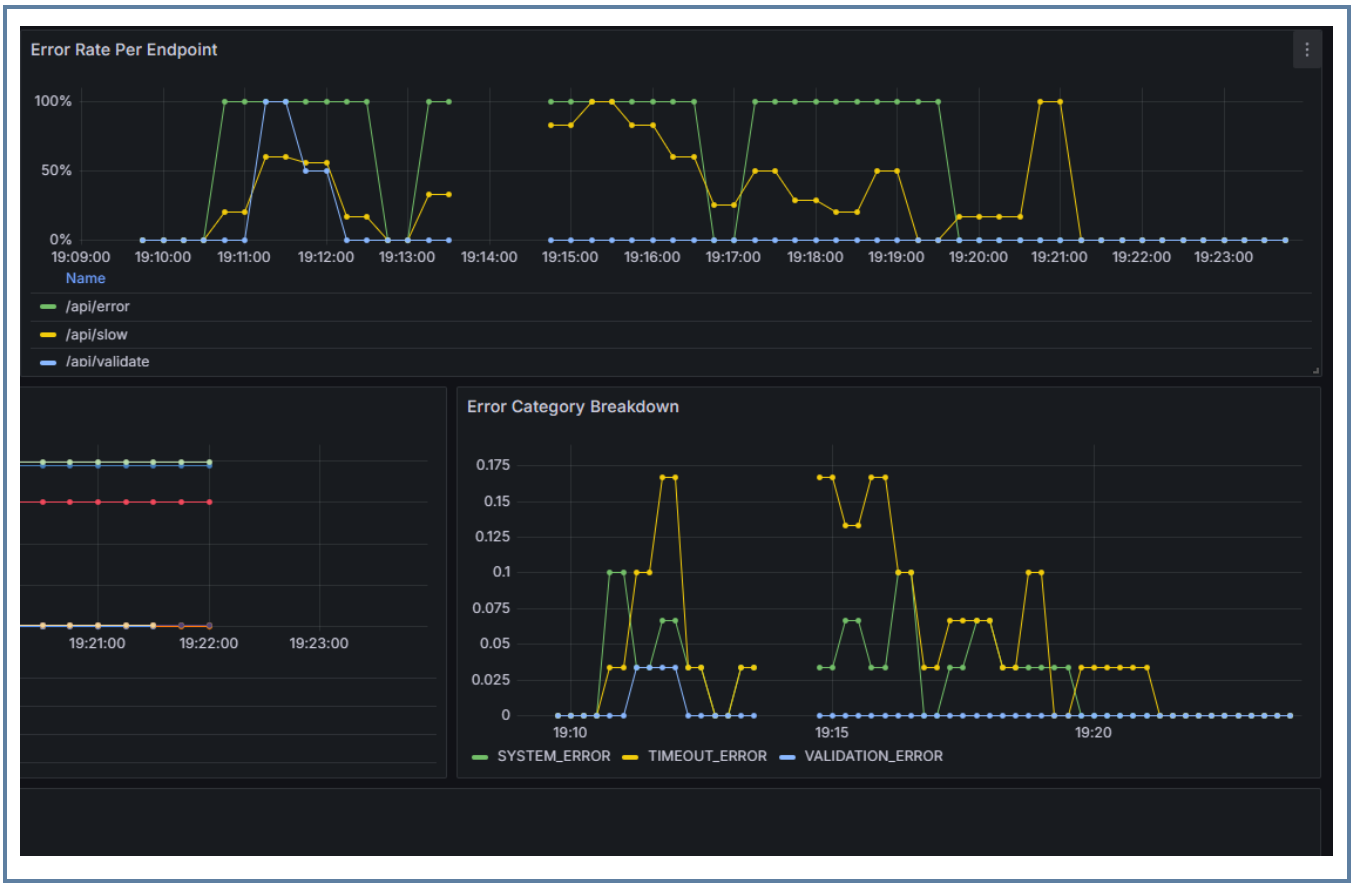
This design is controlled because the start and end times are explicit, and it is visible because it changes both the logs and the dashboard in a way that is operationally obvious.

5. Dashboard Evidence

The Grafana dashboard includes request rate per endpoint, error rate percentage per endpoint, P50/P95/P99 latency per endpoint, error-category breakdown, and an anomaly window marker. The screenshots below are the evidence section for submission.







6. Conclusion

the system now behaves like a compact observability experiment: logs are structured and ML-friendly, metrics are bounded and dashboardable, and the anomaly is controlled, timestamped, and externally documented. The most important design decision was treating latency-only degradation as a first-class error condition. That makes the telemetry more representative of real production incidents than a system that only records success versus failure by status code.